

Insight Ladder — Day 5

Where should the split go?

Priyanka | September 25, 2026 | Easy relative to this week | 120 minutes

Main problem

Split an array of nonnegative integers into two nonempty contiguous parts. Return $(t, \text{difference})$, where the left part contains the first t entries and difference is the absolute difference between the two part sums. Break ties by choosing the smallest t . Input: $2 \leq n \leq 200,000$; entries are at most 1,000,000,000. Require $O(n)$ total time; $O(1)$ auxiliary space is achievable. Explain also whether the candidate split positions have an order property that permits searching after preprocessing.

Examples and checks: $[1,2,3,4]$ gives $(3,2)$. $[0,0,0]$ gives $(1,0)$. $[1,1,1]$ gives $(1,1)$. Test $n=2$ and zero runs, including repeated equally good splits.

Optional variation

Allow negative entries with absolute value at most 1,000,000,000. Give a counterexample to the order property used for searching. Then provide a correct $O(n)$ -time, $O(1)$ -space algorithm. For $[4,-5,3,2]$, the answer is $(3,0)$. Does your original linear scan need any change?

Before reading hints: Express both part sums using one total and one running quantity. Would building an auxiliary structure and then searching improve total time for this single query? Explain ties carefully.

Your 120 minutes

- 0–10: closed-notes recall of one invariant or failed assumption from the previous session.
- 10–100: solve the main problem. Write a direct solution, identify repeated work, state a claim, try to break it, then implement. Attempt the optional variation only when the main solution is secure; it stays inside these 90 minutes.
- 100–120: stop coding, explain correctness and cost, review evidence, and log partial work. A full solution read during review does not turn an unfinished timed attempt into a solve.
- Use C11 OR Python 3 consistently. After 15 minutes without a useful new step, record the blocker and request one hint or continue. Record the time and level of every hint or appendix consultation.

Submit your evidence

Submit code, tests, the key claim and its justification, runtime/space, minutes spent, and hints used. Main status: independent solve / assisted solve / partial / unattempted. Insight: independent / after small hint / after method supplied. Track variation coding or reasoning separately; a counterexample is not another code solve. Count a main solve only when working code meets the contract and you can justify it. There is no completion quota.

Solution appendix

Open only after an attempt; record what you read and when. Read one hint at a time. The explanation below supplies the method and cannot count as independent discovery.

Progressive hints

1. Let S be the total and P the left sum. Rewrite the difference without summing the right side again.
2. The objective is $|2P-S|$. As t increases, does P ever decrease under the original assumptions?
3. A running-prefix scan already solves the task in $O(n)$ time and $O(1)$ space. A precomputed prefix array permits searching near half the total, but building it still costs $O(n)$.

Main solution reasoning

Compute total S . Scan $t=1$ through $n-1$, adding $a[t-1]$ to P and evaluating $|2P-S|$. Retain a split only when its difference is strictly smaller than the best, preserving the earliest tie. This gives $O(n)$ time and $O(1)$ space. With nonnegative input, P is nondecreasing, so $2P \geq S$ is a monotone predicate on valid split positions. In a precomputed prefix array, the first crossing and the preceding prefix value suffice to determine the smallest difference; if that preceding value repeats, choose its earliest index for the required tie. A plain scan avoids that extra tie handling. Building the prefix array costs $O(n)$ time and $O(n)$ space, so $O(\log n)$ search alone is not the total cost. Do not sort: contiguity belongs to the original order. Use 64-bit sums and absolute differences in C .

Variation review

For $[4,-5,3,2]$, $S=4$ and the valid prefix sums are $4,-1,2$. The predicate $2P \geq S$ is $\text{true}, \text{false}, \text{true}$, so a binary-search boundary argument is invalid. The linear scan makes no monotonicity assumption and still returns $(3,0)$. Changing the input assumptions breaks the search proof but leaves the direct running-sum method correct.

Exit ticket and repair

Express both part sums using one total and one running quantity. Would building an auxiliary structure and then searching improve total time for this single query? Explain ties carefully.

If the explanation is still unclear: List every valid split, prefix sum, and difference for $[0,0,0]$ and $[4,-5,3,2]$.

Log: main attempted __; independent solve __; assisted solve __; partial __; variation status __; insight assistance __; minutes to first useful claim __; total minutes __.